Logestechs Training Hub — Task Manager

Kotlin Console Application | Videos 5–15

Project Idea

Build a small console program in **Kotlin** to manage a to-do list. The user can add tasks, view them, search by ID, mark a task as done, delete a task, and see a short summary. No Android UI, database, or network is needed at this stage; keep the data in memory. The project is designed so it can later grow into an Android app.

Required Features

1. **Add task:** ID, title, category (e.g. Study, Home), priority (1 = low, 2 = medium, 3 = high), and type (work or personal).
2. **Show all tasks:** one line each, with the ID, title, category, priority, and whether it is done.
3. **Search by ID:** show the task if found, otherwise a clear message.
4. **Mark task as done:** by ID.
5. **Delete task:** by ID.
6. **Summary:** total tasks, done tasks, pending tasks, number of tasks per priority, and number of tasks per category.
7. **Exit:** the menu repeats until the user chooses 0.

Main Menu

```
===== Task Manager =====
1. Add Task          4. Mark Task as Done
2. Show All Tasks    5. Delete Task
3. Search by ID      6. Summary
0. Exit
Enter your choice:
```

Required Concepts

Concept (video)	Where to use it
Variables & Data Types (5)	`val`/`var` with `Int`, `String`, `Boolean` for task data (`isDone` is a `Boolean`).
Arrays (6)	An `IntArray(3)` that counts the tasks of each priority level.
Lists (7)	A `MutableList<Task>` to add, remove, and search tasks.
Sets & Maps (8)	A `MutableSet<Int>` of used IDs (to block duplicates); a `MutableMap<String, Int>` for the number of tasks per category.
if/else & when (9)	Validate input with `if`; handle the menu choice with `when`; convert a priority number to a word (Low, Medium, High) with `when`.
Loops (10)	`while` for the menu, `for` to print and count tasks, and `break`/`continue` (e.g. stop the search when found).
Functions (11)	Small functions with parameters and return values, e.g. `findById(id: Int): Task?`.
Classes & Objects (12–13)	A `Task` class with properties and functions; create objects from it.
Constructors & Init (13)	Primary constructor for the data; an `init` block that checks the title is not empty and the priority is between 1 and 3.
Inheritance (14)	`abstract class Task`, with `WorkTask` and `PersonalTask` extending it (`WorkTask` has an extra `deadlineDay` property from 1 to 31).
Abstract & Interfaces (15)	An `abstract fun typeLabel()` that returns "Work" or "Personal"; an `interface Printable` with `fun display()`.
Data Class (15)	`data class Summary(total, done, pending)` returned by the summary function.

You must be able to explain why you used each concept. Do not use inheritance or interfaces without a clear reason.

Delivery Requirements

- Reject a duplicate ID, an empty title, a priority outside 1–3, and an unknown ID, with clear messages.
- The program must not crash when text is entered instead of a number (use `toIntOrNull()`).
- Split the code into small functions (not everything in `main()`).
- Add at least 6 sample tasks (both types), and test error cases (unknown ID, duplicate ID, marking a task done twice).

Future Android Extension (not required now)

The same idea maps directly to an Android app later: the `Task` classes become the data model, the `MutableList` becomes a database (Room) with a list screen (RecyclerView), the console input becomes an "Add task" form, the "done" flag becomes a checkbox, and the summary becomes a screen with totals.

Good luck with the project!